PRACTICAL -2

Title : Build and Train a Feedforward Neural Network (MLP) using TensorFlow

Aim of the Practical

To build and train a Feedforward Neural Network, also known as a Multi-Layer Perceptron (MLP), using TensorFlow and the Fashion-MNIST dataset, tune the hidden layers and activation functions, monitor training and validation performance using accuracy and loss curves, and evaluate the final model using test accuracy.

Theory

A Feedforward Neural Network (FNN) is a type of artificial neural network in which information moves in one direction, from the input layer through one or more hidden layers to the output layer. An MLP is a feedforward network containing fully connected (Dense) layers.

In this practical, the Fashion-MNIST dataset is used. It contains 28 × 28 grayscale images of fashion items belonging to 10 classes. Each pixel has a value from 0 to 255. The pixel values are normalized to the range 0 to 1 by dividing them by 255.0. Normalization helps the neural network train more efficiently.

The Flatten layer converts each 28 × 28 image into a one-dimensional vector of 784 values. The network then uses three hidden Dense layers containing 256, 128, and 64 neurons. ReLU activation is used in the first two hidden layers, while tanh is used in the third hidden layer as part of activation-function tuning. The output layer contains 10 neurons with softmax activation, producing a probability distribution over the 10 Fashion-MNIST classes.

The model is compiled using the Adam optimizer with a learning rate of 0.001 and sparse categorical cross-entropy as the loss function. Adam adjusts the network weights during training, while sparse categorical cross-entropy is suitable because the class labels are represented as integer values. Accuracy is used as the performance metric.

During training, 20% of the training data is used for validation. Accuracy and loss values for both training and validation data are stored in the training history and plotted against epochs. These curves help identify learning behavior and possible overfitting.

Algorithm

1. Start the program and import the required TensorFlow, Matplotlib, and Keras modules.

2. Load the Fashion-MNIST dataset using tf.keras.datasets.fashion_mnist.

3. Split the dataset into training images and labels (x_train, y_train) and test images and labels (x_test, y_test).

4. Normalize the image pixel values by dividing x_train and x_test by 255.0 so that pixel values lie between 0 and 1.

5. Create a Sequential neural network model.

6. Add a Flatten layer to convert each 28 × 28 image into a 784-element one-dimensional vector.

7. Add the first hidden Dense layer with 256 neurons and ReLU activation.

8. Add the second hidden Dense layer with 128 neurons and ReLU activation.

9. Add the third hidden Dense layer with 64 neurons and tanh activation. This layer represents the tuned activation function.

10. Add an output Dense layer with 10 neurons and softmax activation for the ten Fashion-MNIST classes.

11. Compile the model using the Adam optimizer with a learning rate of 0.001, sparse categorical cross-entropy loss, and accuracy as the evaluation metric.

12. Train the model for 10 epochs with a batch size of 32. Use 20% of the training data for validation.

13. Store the training history containing training and validation accuracy and loss values.

14. Evaluate the trained model on the test dataset to obtain test loss and test accuracy.

15. Plot training accuracy and validation accuracy against the number of epochs.

16. Plot training loss and validation loss against the number of epochs.

17. Display the plots and the final test loss and test accuracy.

18. Stop the program.

Model Architecture Used

Layer	Neurons / Shape	Activation
Input / Flatten	$28 \times 28 \rightarrow 784$	—
Hidden Layer 1	256	ReLU
Hidden Layer 2	128	ReLU
Hidden Layer 3	64	tanh
Output Layer	10	Softmax

Result

The final test loss and test accuracy are obtained by running the given TensorFlow program. The accuracy and loss plots provide a visual representation of the model's training and validation performance.

Conclusion

The Fashion-MNIST dataset was successfully used to build and train a Feedforward Neural Network (MLP) using TensorFlow. The images were normalized before training, and the model used three hidden Dense layers with ReLU and tanh activation functions. The Adam optimizer and sparse categorical cross-entropy loss were used to train the network for 10 epochs. Training and validation accuracy and loss curves were plotted to observe the learning behavior of the model. Finally, the trained model was evaluated on the unseen test dataset to determine its generalization performance. The practical demonstrates how hidden-layer structure, activation functions, optimization, and validation monitoring can be combined to develop a neural-network classifier for image data.